

# Charotar University of Science and Technology (CHARUSAT)

Faculty of Technology and Engineering (FTE)

## CO–PO–PSO Mapping Matrix with Justification

|                 |                                                      |
|-----------------|------------------------------------------------------|
| Subject         | Object Oriented Programming (CEUE203)                |
| Semester        | 3rd                                                  |
| Teaching Scheme | Theory 2 hrs/week · Practical 2 hrs/week · 4 Credits |
| Academic Year   | 2026–27                                              |

### 1. Course Outcomes (COs)

**CO1:** Demonstrate a comprehensive understanding of Java fundamentals, including syntax, data types, control structures, and object-oriented programming principles such as inheritance, polymorphism, abstraction, and encapsulation.

**CO2:** Utilize inheritance, interfaces, and Java package structures to create modular and maintainable code, promoting code reusability and efficient project organization while adhering to object-oriented design principles.

**CO3:** Construct and handle exceptions proficiently to build fault-tolerant applications, applying exception handling mechanisms to manage errors and maintain program stability.

**CO4:** Develop and implement multi-threaded applications using Java's built-in thread handling mechanisms and concurrency utilities to optimize program performance and resource utilization.

**CO5:** Design and implement Java programs using file handling, the NIO.2 package, and Stream API to perform efficient file operations and process data streams effectively, enhancing data-driven application capabilities.

**CO6:** Apply the concepts of the Collection Framework and Generics to create robust and efficient programs that effectively manage and manipulate data, ensuring type safety and reducing runtime errors.

### 2. Course Articulation Matrix

The matrix below records the correlation of each Course Outcome with the Programme Outcomes (PO1–PO12) and Programme Specific Outcomes (PSO1–PSO2).

|     | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 | PSO1 | PSO2 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|
| CO1 | 2   | 3   | 3   | -   | -   | -   | -   | -   | -   | -    | -    | -    | -    | -    |
| CO2 | 3   | 2   | 3   | 2   | 1   | 1   | -   | -   | -   | -    | -    | -    | 1    | 3    |
| CO3 | -   | 2   | 2   | 3   | 2   | -   | -   | -   | -   | -    | 1    | 1    | -    | -    |
| CO4 | 1   | 1   | 1   | 1   | 3   | -   | -   | -   | -   | -    | 2    | 2    | 3    | -    |
| CO5 | 1   | 1   | 2   | 3   | 3   | -   | -   | 1   | -   | -    | 1    | 1    | 2    | 2    |
| CO6 | 1   | 1   | 1   | 2   | 3   | 1   | -   | 1   | -   | -    | 2    | 1    | 3    | 3    |

Correlation levels — 1: Slight (Low) · 2: Moderate (Medium) · 3: Substantial (High). Where there is no correlation, “-” is placed.

### 3. Justification of Mapping

Each mapped cell is justified below on the basis of the course content and its intended learning. Programme Outcomes shown with “—” in the matrix are not directly addressed by the corresponding CO and are therefore omitted here.

#### CO1 — Java fundamentals and OOP principles

| Mapped PO/PSO | Level | Justification                                                                                                                                                                                |
|---------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PO1           | 2     | Applying Java syntax, data types, and OOP fundamentals draws on engineering-specialization knowledge, but at a foundational rather than a deeply specialized level, so the link is moderate. |
| PO2           | 3     | Reasoning about how control structures and OOP constructs behave requires identifying and formulating program logic from first principles — a strong problem-analysis link.                  |
| PO3           | 3     | Encapsulation, abstraction, and inheritance are inherently design constructs; understanding them directly builds the ability to structure solution components.                               |

#### CO2 — Inheritance, interfaces, and packages for modular code

| Mapped PO/PSO | Level | Justification                                                                                                                                                    |
|---------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PO1           | 3     | Strong application of core OOP engineering knowledge — inheritance, interfaces, and package/module structures.                                                   |
| PO2           | 2     | Deciding class hierarchies and interface contracts involves moderate analysis of how to decompose a system.                                                      |
| PO3           | 3     | Producing modular, maintainable, and reusable code is the essence of designing solution components.                                                              |
| PO4           | 2     | Comparing design alternatives (inheritance vs. composition, package/module organization) involves moderate investigation of trade-offs.                          |
| PO5           | 1     | Slight use of modern tooling — build/packaging utilities, JAR files, and the Java module system.                                                                 |
| PO6           | 1     | Writing maintainable, well-organized code carries a slight professional-practice dimension toward those who later use or extend it.                              |
| PSO1          | 1     | Slight contribution to analytical and technical IT-solution skills through structured design thinking.                                                           |
| PSO2          | 3     | Using interfaces and modules to design modular IT-based solutions that meet specified needs maps strongly to designing solutions with contemporary technologies. |

#### CO3 — Exception handling for fault-tolerant applications

| Mapped PO/PSO | Level | Justification                                                                                                                                              |
|---------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PO2           | 2     | Identifying and formulating the error and failure scenarios a program must handle is moderate problem analysis.                                            |
| PO3           | 2     | Designing try-catch-finally structures, custom exceptions, and propagation strategies moderately develops robustness-oriented design.                      |
| PO4           | 3     | Reproducing, investigating, and interpreting failure conditions, and validating that exception paths behave correctly, is strongly investigation-oriented. |

| Mapped PO/PSO | Level | Justification                                                                                                      |
|---------------|-------|--------------------------------------------------------------------------------------------------------------------|
| PO5           | 2     | Moderate use of modern tools — debuggers, try-with-resources, and IDE diagnostics to trace and resolve exceptions. |
| PO11          | 1     | Fault tolerance contributes slightly to project and quality management by reducing failures over a system's life.  |
| PO12          | 1     | Slight lifelong-learning link, as robust error-handling practice transfers to new APIs and frameworks.             |

#### CO4 — Multithreading for optimized performance

| Mapped PO/PSO | Level | Justification                                                                                                                                            |
|---------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| PO1           | 1     | Slight engineering-knowledge link — concurrency is a specialized topic drawing less directly on foundational mathematics and science.                    |
| PO2           | 1     | Slight problem-analysis contribution in reasoning about concurrent execution.                                                                            |
| PO3           | 1     | Slight design contribution through structuring thread interactions.                                                                                      |
| PO4           | 1     | Slight investigation link in observing thread behaviour, race conditions, and deadlocks.                                                                 |
| PO5           | 3     | Strong modern-tool usage — Thread/Runnable, Executor and ExecutorService thread pools, and the Fork/Join framework are applied directly.                 |
| PO11          | 2     | Managing thread pools and optimizing resource utilization maps moderately to engineering and management principles for workload and resource management. |
| PO12          | 2     | Concurrency is an advanced, evolving area that moderately requires and reinforces independent, continued learning.                                       |
| PSO1          | 3     | Optimizing performance and resource use through concurrency is a direct application of computational thinking to produce optimized IT solutions.         |

#### CO5 — File handling, NIO.2, and the Stream API

| Mapped PO/PSO | Level | Justification                                                                                                                                                 |
|---------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PO1           | 1     | Slight foundational-knowledge link; the emphasis is on applied I/O APIs rather than core theory.                                                              |
| PO2           | 1     | Slight problem-analysis contribution in framing data-processing requirements.                                                                                 |
| PO3           | 2     | Designing file-handling and stream-processing pipelines moderately develops solution-design ability.                                                          |
| PO4           | 3     | Reading, transforming, and interpreting data through streams and analyzing I/O results aligns strongly with investigation — data analysis and synthesis.      |
| PO5           | 3     | Strong modern-tool usage — the NIO.2 package, Path/Files APIs, buffers and channels, and the Stream API are applied directly.                                 |
| PO8           | 1     | Handling files and data responsibly (integrity, correct resource closing) carries a slight professional-ethics dimension.                                     |
| PO11          | 1     | Slight project-management link through efficient handling of data resources.                                                                                  |
| PO12          | 1     | Slight lifelong-learning link as I/O APIs continue to evolve.                                                                                                 |
| PSO1          | 2     | Moderate contribution to analytical and technical IT-solution skills for data-driven applications.                                                            |
| PSO2          | 2     | Using NIO.2 and the Stream API to build data-driven IT solutions that meet specified processing needs is a moderate application of contemporary technologies. |

## CO6 — Collection Framework and Generics

| Mapped PO/PSO | Level | Justification                                                                                                                                    |
|---------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| PO1           | 1     | Slight foundational-knowledge link; the focus is applied use of the Collections and Generics APIs.                                               |
| PO2           | 1     | Slight problem-analysis contribution in framing data-management requirements.                                                                    |
| PO3           | 1     | Slight design contribution in structuring data storage.                                                                                          |
| PO4           | 2     | Selecting and evaluating appropriate collections (List/Set/Map trade-offs) for a dataset involves moderate investigation.                        |
| PO5           | 3     | Strong modern-tool usage — the Collection Framework and Generics are core contemporary Java facilities applied directly.                         |
| PO6           | 1     | Slight professional-practice dimension in delivering robust, reliable software.                                                                  |
| PO8           | 1     | Ensuring type safety and reducing runtime errors carries a slight reliability and ethics responsibility.                                         |
| PO11          | 2     | Efficient data management with the right structures maps moderately to engineering and management principles for maintainable projects.          |
| PO12          | 1     | Slight lifelong-learning link as generics and collections underpin further study.                                                                |
| PSO1          | 3     | Choosing optimal, type-safe structures for efficient data handling is a strong application of computational thinking to optimized IT solutions.  |
| PSO2          | 3     | Building robust, efficient, data-management IT solutions that meet specified needs maps strongly to designing solutions with these technologies. |